

Глава 5 Обобщение – Невронни мрежи за игри с непълна информация

Изследване върху възможностите за прилагане на изкуствен интелект в компютърните игри

Alexander Andreev

May 2026

Съдържание

5 Глава 5 – Невронни мрежи за игри с непълна информация	1
Защо невронни мрежи	1
5.1 Основни положения	2
5.2 Архитектура и композиция	5
5.3 Невронни мрежи в CFR	13
5.4 Други невронни приложения	18

5 Глава 5 – Невронни мрежи за игри с непълна информация

Това е съкратен обзор на невронни методи за игри с непълна информация – материалът, който Глава 5 разширява от първоначалния си фокус върху Деер CFR до по-широко разглеждане на изграждането, обучението и внедряването на невронни мрежи в игри, при които играчите не могат да възприемат пълното състояние. Той служи като кратък справочник при четенето на следващите глави. Глава 5 представлява умишлено еднократно отклонение от обичайния учебен цикъл – фазата на реализация от нулата се отлага, а вместо нея се разширява теорията – поради което този документ е карта на областта в ширина, а не запис на код. Математическите постановки са представени опростено: ключовите уравнения се посочват там, където изострят интуицията, докато извежданията и доказателствата остават в цитираните източници. Разделите за научената абстракция, оценяването на невронни стратегии, експерименталните архитектури и обобщаващата карта на решенията са планирани, но не са включени в тази версия.

Защо невронни мрежи

Табличният CFR и неговите варианти на Монте Карло (Глави 2–4) съхраняват стойност на съжалението и стойност на стратегията във всяко информационно множество, така че разходът им за памет е пропорционален на броя на информационните множества, който нараства с размера на

играта.¹ Експериментите в тази глава определиха количествено тази граница в Ледюк покер – игра, достатъчно малка, за да се побере в таблица: табличният MCCFR достигна експлоатируемост 0,097, докато Деер CFR при сравнимо време за изпълнение остана над 1,1 при всички изпробвани размери на мрежата. В този мащаб табличната методика е за предпочитане; невронната методика не предлага никакво предимство, тъй като таблицата вече се побира изцяло в паметта.

Невронна мрежа заменя таблицата с апроксимация на функция f_θ , която съпоставя характеристиките на едно състояние към стойност или разпределение на действията и обобщава към състояния, които не са били посетени по време на обучението. Това обръща структурата на разходите: паметта става независима от броя на информационните множества, за сметка на по-скъпи итерации и по-слаби гаранции за сходимост. Двата режима се разделят от праг в размера на играта: под него изброяването е по-евтино и точно, а над него таблицата не може да бъде построена и апроксимацията на функция е единствената възможност. Игрите, към които е насочена тази работа, са над този праг, поради което останалата част от главата се посвещава на невронни методи.

5.1 Основни положения

5.1.1 Основи на невронните мрежи

Мрежа с директно разпространение представлява композиция от слоеве, като всеки слой прилага афинно преобразуване, последвано от нелинейност по елементи:

$$h = \sigma(Wx + b),$$

където x е входът на слоя, W и b са съответно обучаемата матрица на теглата и векторът на отместванията, а σ представлява фиксирана нелинейност, като например изправителя $\text{ReLU}(z) = \max(0, z)$. Подреждането на L такива слоя формира функцията на мрежата f_θ , при която θ обединява всички W и b . Именно нелинейностите позволяват на тази композиция да представя функции, които не могат да бъдат изразени чрез едно-единствено афинно преобразуване.

ReLU (rectified linear unit): стандартната функция за активация – тя пропуска положителните стойности непроменени и ограничава отрицателните до нула, което е изчислително евтино и избягва проблема с изчезващите градиенти, характерен за по-старите насищащи се функции за активация.

Обучението изисква **скаларна величина** за измерване на грешката. При задача за **регресия** – например при оценка на **контрафактична стойност** – стандартно се използва **средноквадратичната грешка**:

¹Zinkevich, M., Johanson, M., Bowling, M. & Piccione, C. (2007). “Regret Minimization in Games with Incomplete Information.” *Advances in Neural Information Processing Systems* 20, 1729-1736.

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N (f_{\theta}(x_i) - y_i)^2,$$

където N е броят на примерите в партида, x_i - вход, а y_i - неговата цел. Когато целта представлява разпределение на действията, вместо това се използва загубата на кръстосана ентропия $-\sum_a y_a \log f_{\theta}(x)_a$, като y_a е целевата вероятност за действие a .

Обучението представлява **градиентен спуск** върху тази **загуба**. Всеки параметър се измества с малка стъпка в посоката, която води до най-голямо намаление на **загубата**:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L,$$

където η е скоростта на обучение, а $\nabla_{\theta} L$ - градиентът на загубата спрямо параметрите. *Обратно разпространение на грешката* изчислява този градиент, като прилага правилото за верижно диференциране слой по слой от изхода към входа и използва повторно междинните стойности, съхранени във всеки слой; така изчисляването на един градиент струва приблизително колкото едно преминаване напред. Автоматичното диференциране извършва тази процедура механично.

На практика градиентът се оценява от малка случайна партида, а не от целия набор от данни (*стохастичен градиентен спуск*) – по-шумен, но значително по-евтин на стъпка. Импулсът намалява този шум, като извършва стъпките с текуща средна на градиентите, $v \leftarrow \beta v + \nabla_{\theta} L$ и $\theta \leftarrow \theta - \eta v$, където $\beta \in [0, 1)$ определя степента, в която се запазват миналите градиенти. Adam, оптимизаторът по подразбиране в методите на тази глава, освен това разделя стъпката за всеки параметър на текуща оценка на големината на неговия градиент, което осигурява стабилен напредък с минимално ръчно настройване на η .

Adam (адаптивна оценка на моментите): комбинира импулс с размери на стъпките за всеки параметър, извлечени от текущи статистики на градиента, поради което се обучава надеждно с малко настройване.

Мрежа с достатъчен капацитет може да напасне точно една тренировъчна партида, включително и нейния шум, и след това да се представи слабо при невиджани състояния – *преобучение* (overfitting). Регуляризацията противодейства на това. Отслабването на теглата добавя наказание $\lambda \|\theta\|^2$ към загубата, като λ контролира степента, в която се обезкуражават големите тегла; отпадането произволно занулява част p от активациите по време на обучението, така че мрежата да не може да разчита на нито един отделен елемент; и ранното спиране прекратява обучението, след като грешката върху задържаните данни започне да нараства.

Една идея от обучението с подкрепление се повтаря през цялата глава и си струва да бъде изразена изрично: *самоподкрепяне* (bootstrapping). Актуализацията чрез темпорални разлики оценява стойността на едно състояние въз основа на собствената оценка на мрежата за следващото състояние, а не от краен резултат:

$$V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)],$$

където s и s' са последователни състояния, r е наградата, $\gamma \in [0, 1]$ е коефициентът на дисконтиране, а α е размерът на стъпката; величината в скобите представлява грешка при темпорална разлика – разликата между текущата оценка и целта със самоподкрепяне. Самоподкрепянето прави обучението ефективно по отношение на данните, а както показва следващият раздел, също така го прави и по-малко стабилно. Основният материал е представен в глави 9 и 11 от Sutton & Barto (2-ро издание); всяка стандартна литература за дълбоко обучение разглежда подробно обратното разпространение на грешката и оптимизацията.

5.1.2 Обединяване на основите в Ледюк покера

Тези елементи се комбинират в най-простия **невронен решаващ** за игра. Вземете Ледюк покер, тестовата среда от глави 3–4: неговото информационно състояние – кой играч е на ход, тайната карта, общата карта и историята на залозите – се кодира като вектор от 30 числа. Малка мрежа съпоставя на този вектор стойност за всяко действие; обратното разпространение на грешката с Adam я настройва спрямо целевите стойности, генерирани по време на игра, а леката **регуляризация** предотвратява запаметяването на шума в тези цели. Използвана правилно, мрежата предоставя две възможности, които таблицата няма: тя обобщава, така че **стойността**, научена в едно **информационно множество**, информира подобни невиджани такива, а цената на паметта ѝ е фиксирана от броя на параметрите, а не от броя на информационните множества.

Същата техника се влошава, когато бъде претоварена, а **изследването** направи режимите на отказ конкретни. Капацитетът е най-ясният пример. При **Ледюк** най-малката тествана **мрежа** – два скрити слоя с тридесет и две единици – достигна по-ниска **експлоатируемост** от мрежа, няколко пъти по-голяма, вероятно защото при толкова малък бюджет по-големите мрежи не получават достатъчно различни извадки, за да се обучат (резултатът е от едно начално число при почти хоризонтални криви, затова обяснението е хипотеза). **Скоростта на обучение**, зададена твърде висока, дестабилизира спускането; прекомерната **регуляризация** изглажда острите, почти **детерминистични** стратегии, към които CFR се сближава, тъй като силното **отслабване на теглата** или **отпадане** дърпат изхода към **равномерното разпределение**. Общият принцип е, че капацитетът и регуляризацията трябва да бъдат съобразени с количеството и качеството на данните, а не да се максимизират по подразбиране – точка, която раздел 5.2 развива в изрични насоки за оразмеряване.

5.1.3 Нестационарност и смъртоносната троица

Комбинирането на горните компоненти въвежда нестабилност. Sutton и Barto идентифицират една **смъртоносна троица**: когато **апроксимация на функция**, **самоподкрепяне** и **обучение извън текущата стратегия (off-policy)** се използват заедно, параметрите могат да се отдалечат неограничено, вместо да клонят към решение, тъй като **целевите стойности по метода**

„bootstrap“ за всяка актуализация зависят от същите параметри, които се променят. Всеки един компонент е **безопасен** самостоятелно; комбинацията не е. Средствата за защита са практични, а не теоретични. Една *целева мрежа* – забавено копие на параметрите, използвано само за изчисляване на **целеви стойности чрез самоподкрепяне** – и един *буфер за възпроизвеждане*, който декорелира последователните проби, заедно правят дълбоките Q-мрежи обучаеми, и двата се появяват отново в невронните решаващи на игри от раздели 5.3 и 5.4.

Игрите въвеждат втори източник на нестабилност, който липсва при обучението на един агент. При обучението с учител разпределението на данните е фиксирано, а при обучението с подкрепление с един агент то се стабилизира, след като стратегията достигне сходимост. В една игра противникът също се адаптира, така че разпределението на ситуациите, които мрежата среща, се променя по време на обучението; функцията, която се апроксимира, сама по себе си е подвижна цел. Тази нестационарност не е случайна – тя е централната трудност на средата, към която това изследване е насочено. Няколко метода в следващите раздели се разбират най-добре като отговор на нея: играта срещу себе си срещу замразени или осреднени във времето противници възстановява стационарна цел, популационните методи заменят подвижната цел с изричен набор от фиксирани противници, а техниките за моделиране на противника и бърза адаптация (глави 7 и 12) се опитват да я проследят пряко.

5.2 Архитектура и композиция

5.2.1 Индуктивно пристрастие: съгласуване на архитектурата със структурата на играта

Всяка архитектура кодира предположения относно структурата на своя вход още преди да види каквито и да е данни; тези вградени предположения представляват нейното *индуктивно пристрастие*. Напълно свързаната мрежа не предполага нищо – всяка входна характеристика може да взаимодейства с всяка друга – което я прави обща, но изисква голям обем данни, тъй като мрежата трябва да научи структурата от нулата. Изборът на архитектура, чието пристрастие съответства на структурата на играта, намалява броя на параметрите и количеството необходими данни за постигане на определена точност, както и подобрява обобщаването към състояния, които не са били част от обучението. Решенията за архитектура в останалата част от раздел 5.2 по същество представляват избор на индуктивно пристрастие.

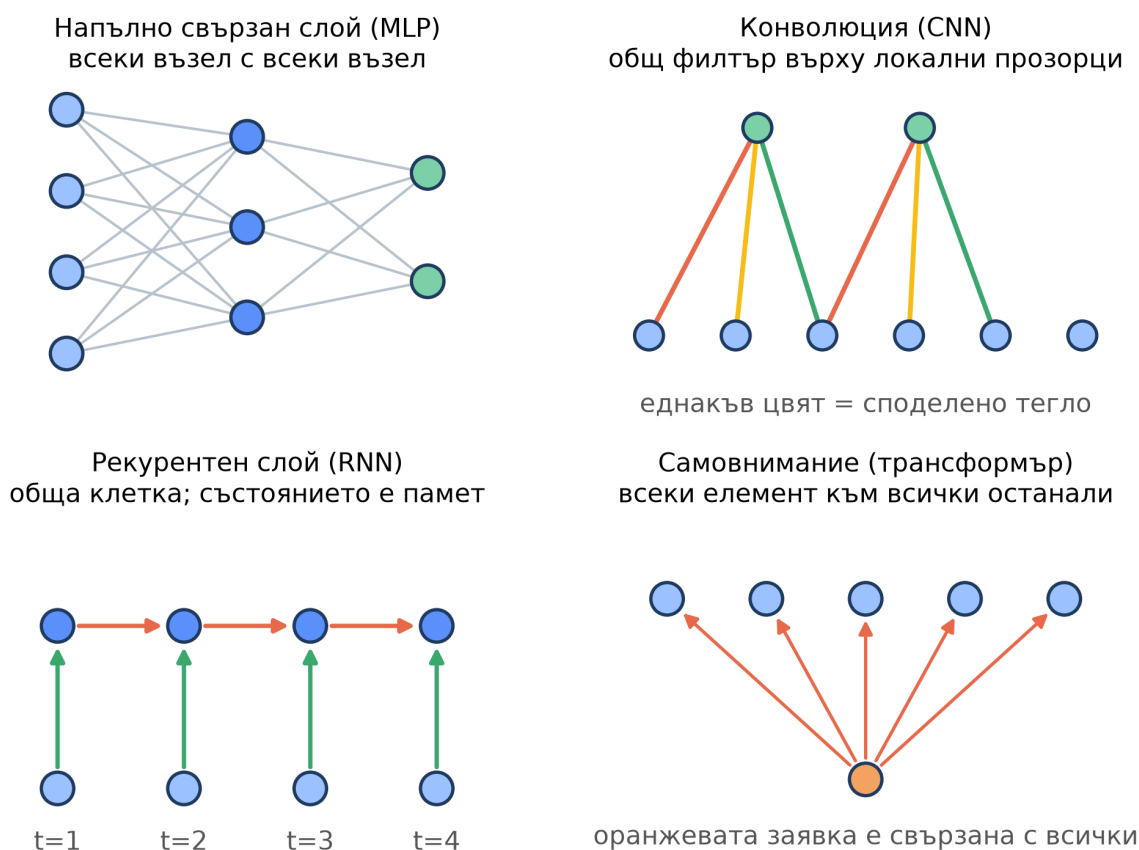
Три съответствия покриват повечето случаи в игри с **непълна информация**. Когато **състоянието** е пространствено – дъска или решетка – **конволюционната мрежа** предполага, че един и същ локален модел е смислен, независимо къде се появи, така че характеристика, научена в една област, може да бъде приложена към други; тази *еквивариантност спрямо трансляция* обяснява защо конволюциите са стандартният избор за дъски и решетки.² Когато част от **състоянието** е неопределена съвкупност – картите в ръката или множеството от останалите играчи – **мрежата** трябва да бъде *инвариантна спрямо пермутации*, като произвежда един и същ изход, независимо от реда, в който тези елементи са представени, за да не се използва капацитет за научаване

²LeCun, Y., Bengio, Y. & Hinton, G. (2015). “Deep learning.” *Nature* 521, 436–444. DOI 10.1038/nature14539.

на това, че редът е без значение. Когато **състоянието** представлява история, разкрита с течение на времето, както при **частична наблюдаемост**, **рекурентна мрежа** или **механизъм за внимание** позволяват на **мрежата** да обобщи последователността и да пренесе напред изводите от по-ранните наблюдения. Съгласуването на тези пристрастия със структурата на играта е разликата между **мрежа**, която се учи ефективно, и такава, която само би могла по принцип.

5.2.2 Типове слоеве на практика: MLP, CNN, RNN, внимание/трансформър

Семействата се различават по своята *свързаност* – кои единици са свързани с кои – и дали свързващите *тегла* се *споделят* между позиции или във времето. Всичко останало (невронът, нелинейността, обратното разпространение на грешката) е идентично. В панела на конволюцията цветът на връзката обозначава теглото: еднаквият цвят означава споделено тегло, а в останалите панели цветът само различава видовете връзки.



Фигура 1: Свързаност на четирите семейства слоеве: напълно свързан слой (MLP), конволюция, която прилага един и същ филтър към локални прозорци, рекурентна клетка, използвана многократно във времето, и самовнимание.

Напълно свързан слой – многослоен перцептрон, или MLP – свързва всяка единица с всяка единица в следващия слой, като всяка връзка носи свое собствено тегло. Той не предполага никаква

структура и може да представи всяка зависимост по принцип, но броят на теглата е произведение от размерите на двата слоя и цялата структура трябва да бъде научена от данните. Това е правилният избор за плосък вектор на характеристики без пространствено или последователно значение, като например информационно състояние в покер.

Конволюционният слой (CNN) свързва всяка изходна единица само с малък локален прозорец от входа – неговото *приемно поле* – и прилага един и същ малък набор от тегла, *филтърът*, на всяка позиция. От тази една промяна в свързаността следват две последствия: броят на теглата зависи от размера на филтъра, а не от размера на входа, и модел, научен на едно място, се разпознава на всяко друго място (еквивариантност спрямо трансляция). Наслагването на няколко конволюционни слоя увеличава ефективното приемно поле, така че по-късните слоеве комбинират локални модели в по-големи. Това е стандартният избор за дъски и решетки.

Споделяне на тегла: повторно използване на едни и същи тегла на много позиции (CNN) или времеви стъпки (RNN). Това е, което прави тези подредби параметърно-ефективни и е механизмът зад тяхната еквивариантност.

Рекурентният слой (RNN) прилага една и съща клетка последователно по дължината на последователността: на всяка стъпка той приема текущия вход заедно със *скритото състояние* от предишната стъпка и произвежда ново скрито състояние, което се предава напред. Тъй като клетката е споделена между стъпките, броят на теглата не зависи от дължината на последователността, а скритото състояние функционира като памет за всичко видяно до момента – свойството, необходимо когато състоянието представлява история, разкривана с течение на времето, както при частична наблюдаемост. Вариантите с гейтове (LSTM, GRU) добавят научени гейтове, които контролират какво да се запази и какво да се изхвърли, което им позволява да задържат информация за по-дълги интервали от време.

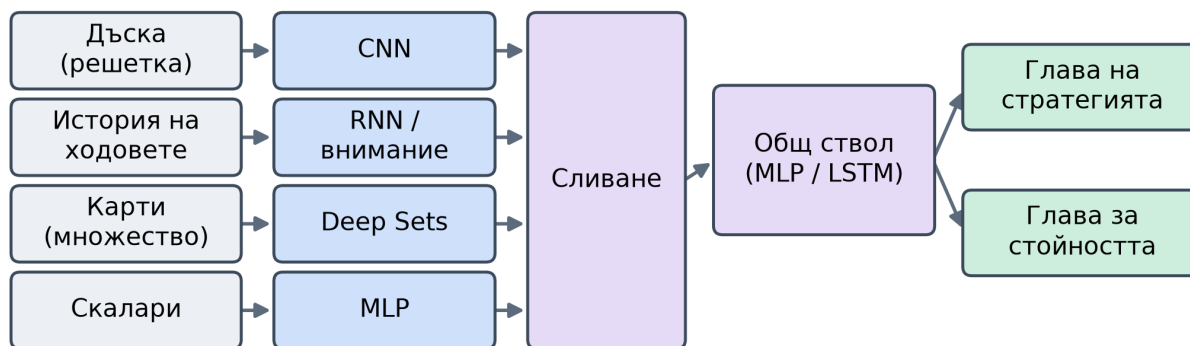
Слой за **самовнимание** (трансформър) позволява на всеки елемент да взаимодейства директно с всеки друг. Всеки елемент излъчва *заявка*, *ключ* и *стойност*; изходът за даден елемент представлява претеглена средна стойност на стойностите на всички елементи, като теглата се определят от сходството между неговата заявка и всеки ключ. Тъй като операцията не отчита реда на елементите, тя е еквивариантна спрямо пермутации (пренареждането на входа пренарежда изхода по същия начин), а редът трябва да бъде подаден изрично чрез **позиционни кодировки**, когато има значение. Самовниманието улавя **дългосрочни зависимости**, до които рекурентният слой достига само косвено, за сметка на изчисления, чиято сложност нараства квадратично спрямо броя на елементите. То е подходящо за истории с променлива дължина и входове със стойности от множества.

Вход, който е наистина неподреден – например картите в ръката или множеството от останалите играчи – изисква **инвариантност спрямо пермутации**, която не се постига чрез нов тип слой, а чрез подредба: прилага се една и съща малка мрежа към всеки елемент, след което резултатите се комбинират с операция, независима от реда, като сума, **средна стойност** или максимум. Това е принципът зад Deep Sets и представлява естественият **кодировчик** за колекция, чийто ред няма

значение.

На практика една мрежа рядко използва само едно семейство архитектури. Обичайният дизайн е конвейер: отделен *кодировчик*, съобразен с всяка част от входа, изходите на кодировчика се *обединяват* чрез конкатенация в *общ ствол*, и една или повече *глави*, които извличат прогнозите от ствола – обикновено глава на стратегията и стойностна глава. Deep Recurrent Q-Network (DRQN) комбинира конволюционен кодировчик, LSTM и изходен MLP, като рекурентността осигурява памет, която едно преминаване напред няма при частична наблюдаемост – естественият шаблон за агент в решетъчна среда с „мъгла на войната“.³ AlphaStar отива по-далеч: кодира множеството от единици с трансформър, пространствената миникарта с CNN и скаларните величини с MLP, след което обединява трите представяния и ги подава през LSTM ядро към изходните глави за действия.⁴ Практическият урок за останалата част от раздел 5.2 е, че „коя архитектура“ обикновено означава „кой кодировчик за всеки вход и как да ги комбинираме“, а не един-единствен избор.

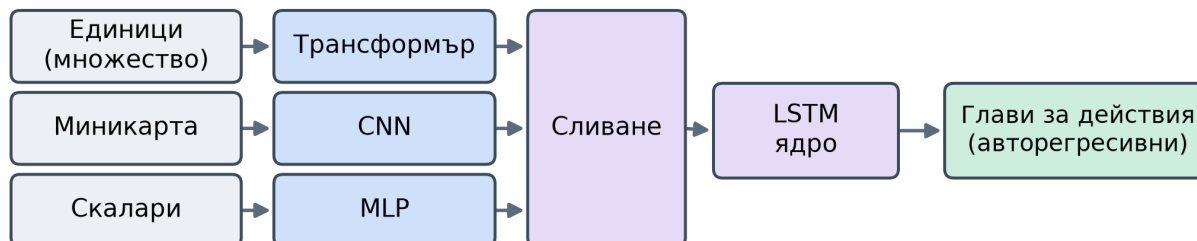
Обща схема: кодировчици → сливане → общ ствол → изходни глави



Пример 1 — DRQN (памет при частична наблюдаемост)



Пример 2 — AlphaStar



Фигура 2: Общата схема с кодировчици, общ ствол и изходни глави (горе) и два нейни примера (долу): DRQN и AlphaStar.

³Hausknecht, M. & Stone, P. (2015). “Deep Recurrent Q-Learning for Partially Observable MDPs.” *arXiv:1507.06527*.

⁴Vinyals, O. et al. (2019). “Grandmaster level in StarCraft II using multi-agent reinforcement learning.” *Nature* 575, 350–354. DOI 10.1038/s41586-019-1724-z.

5.2.3 Кодиране на състоянието на играта и историята

Преди някоя от тези архитектури да може да работи, състоянието на играта трябва да бъде превърнато в числа. Информационното състояние – всичко, което действащият играч може законно да наблюдава – трябва да стане тензор с фиксирана форма, а начинът, по който е подредено, определя коя архитектура може да го използва. Категориалните факти се кодират или като *one-hot* вектори – вектор с дължина k , съдържащ една единствена единица, която маркира коя от k категории е валидна – или, когато k е голямо, като *вграждания*: кратки научени вектори, които се извличат за всяка категория и се обучават заедно с мрежата. Ледюк покер е достатъчно малък, за да използва *one-hot* навсякъде: играчът на ход, тайната карта, общата карта и историята на залозите стават *one-hot* полета, което дава трийсетмерния вектор на OpenSpiel; размерът на пота не се подава отделно, защото следва от историята на залозите.

One-hot срещу вграждане: *one-hot* е точен, но неговата дължина нараства с броя на категориите; едно вграждане компресира голям речник – всяка карта или всеки елемент от дъската – в няколко научени измерения и позволява на подобни категории да споделят структура.

По-богатите входове изискват по-богати подредби. Пространственото състояние се кодира като поредица от наслоени решетки, по една *канална* равнина за всеки тип обект, така че конволюцията да може да го обработи пряко. Pommerman – игра в стил Bomberman върху решетка, в която агентите се движат по дъска, поставяйки бомби, за да разрушават стени и да прихващат противници – е ясен пример: неговата дъска 11×11 е разделена на отделна двоична равнина за всеки тип обект (проходи, дървени стени, твърди стени, бомби, пламъци, бонуси, действащият агент и всеки противник), подредени в тензор (11, 11, канали); скаларни величини като оставащите боеприпаси и обхватът на взрива след това се добавят като кратък вектор или се излъчват като постоянни равнини. История, чийто ред има значение, се кодира като *последователност* от вектори за всяка стъпка за рекурентен слой или трансформър, вместо да бъде сплескана в един вектор, който пренебрегва реда. Непоредена колекция, като текущия набор от противници, се кодира елемент по елемент и се обединява. Във всеки случай кодирането е мястото, където индуктивното приращение от предишните раздели всъщност се прилага: решетката се подава като решетка именно за да са в сила предположенията на конволюцията.

Една дисциплина е специфична за игри с **непълна информация** и лесно може да бъде нарушена: кодирането трябва да съдържа само това, което играчът може да наблюдава, и нищо скрито. Ако каквато и да е **привилегирована информация** – **тайната карта** на противника или истинското съдържание на клетка, покрита с мъгла – попадне в тензора за вход, мрежата ще се научи да я използва и ефективно ще „вижда“ през мъглата, което води до стратегия, която не може да бъде приложена срещу реален противник, и до показател за **експлоатируемост**, който е безсмислен. Ето защо рамки като OpenSpiel дефинират изрично представяне на **информационно състояние** за всеки играч; правилното получаване на това представяне е изискване за коректност, а не избор

за настройка.⁵

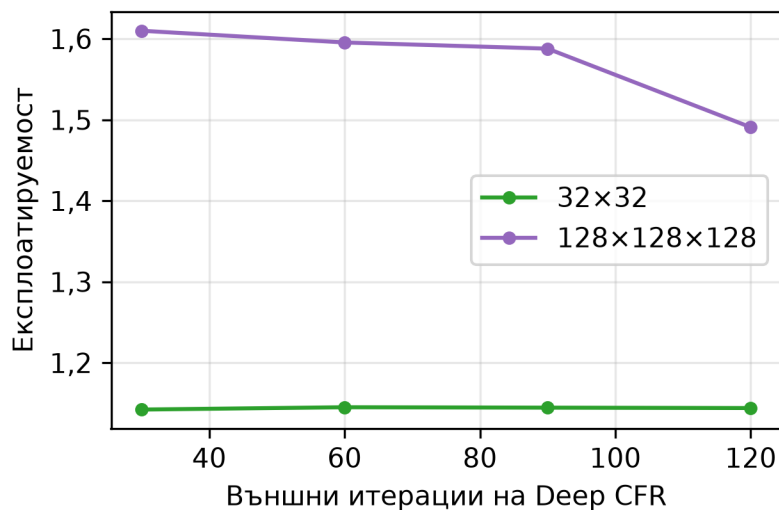
Остават две по-дребни практически бележки. Непрекъснати признаци като **пота** или **размера на стака** се нормализират до сравним мащаб, преди да постъпят в мрежата, така че нито един признак да не доминира градиента. А кодировчикът не е задължително да бъде проектиран ръчно от край до край: една мрежа може да се научи да компресира дълга история в компактен обобщен вектор – идея, която стои в основата на научените абстракции и на научените състояния на убеждение (глава 6).

5.2.4 Оразмеряване и капацитет

Капацитетът на една *мрежа* – определен от нейната ширина, броя на единиците във всеки слой, и нейната дълбочина, броя на слоевете – е размерът на семейството от функции, които тя може да представи. При твърде малък капацитет мрежата остава *недообучена* (underfitting) и не може да изрази целевата стратегия; при твърде голям капацитет тя се *преобучава* (overfitting), като напасва шума в ограничения набор от данни вместо основната структура. Правилният капацитет е този, който съответства на количеството информативни данни, което процедурата за обучение действително предоставя – а при решавач на игра това често е значително по-малко от суровия брой състояния, тъй като самите стойности, които се пасват, представляват шумни оценки.

Експериментите в тази глава показаха това пряко. Прилагайки Deep CFR върху Ледюк при фиксиран бюджет, но с различни размери на мрежата, най-малката мрежа – два скрити слоя от тридесет и две единици – постигна по-ниска експлоатируемост (1,14) от трислойната мрежа, четири пъти по-широка (1,49). Вероятното обяснение е, че няколкото хиляди обхождания не стигат, за да се обучат по-големите мрежи; при едно начално число и почти хоризонтални криви това е хипотеза, а не измерен ефект. Трети опит, с два слоя от по шестдесет и четири единици, е изключен: той остава на 1,70 във всяка контролна точка – стойността, която решаващът даваше преди поправката на грешката му (раздел 5.2.5), затова изчаква повторно изпълнение. Изводът не е, че по-малкият размер винаги е по-добър, а че капацитетът трябва да съответства на бюджета за данни: при по-голяма игра или с много повече обхождания класирането вероятно би се обърнало.

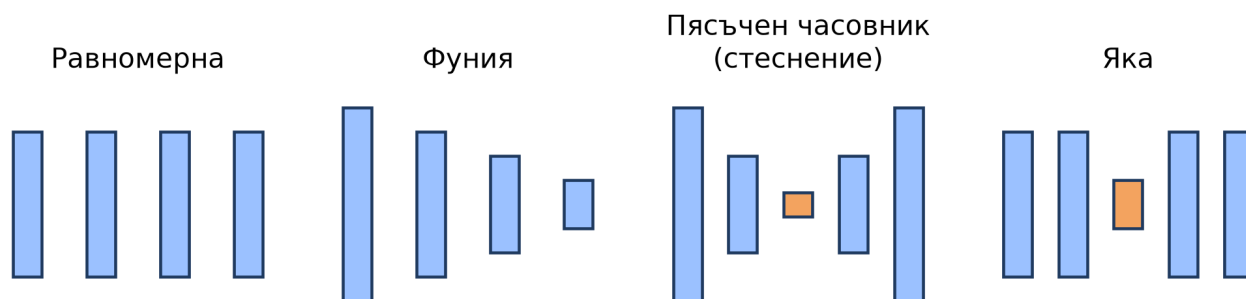
⁵Lanctot, M. et al. (2019). “OpenSpiel: A Framework for Reinforcement Learning in Games.” *arXiv:1908.09453*.



Фигура 3: Експлоатируемост на Deep CFR в Ледюк при два размера на мрежата (32×32 и 128×128×128; едно начално число). Опитът с 64×64 е изключен до повторното му изпълнение.

Това е аргумент в полза на съзнателното оразмеряване на мрежата, вместо да се използва по подразбиране най-голямата, която се побира в паметта. Работеща рецепта е да се започне с малък размер и ширината и дълбочината да се увеличават само докато задържан показател – загуба при валидиране или експлоатируемост за решавач – продължава да се подобрява, като се спира, когато достигне плато. Ширината позволява на мрежата да представя повече различни модели на дадено ниво на абстракция; дълбочината ѝ позволява да съставя прости модели в по-абстрактни, поради което пространствените и последователните проблеми се възнаграждават с няколко слоя, докато плосък вектор на характеристики често не. Капацитетът трябва да се мащабира спрямо размера и богатството на играта, а не да бъде максимизиран по подразбиране.

Ширината на мрежата не е задължително да остава постоянна във всичките ѝ слоеве. Те могат да се *свиват* към изхода, като компресират входа до малко обобщение (кодировчик или *фуния*), или да се *разширяват* от малък вход, за да реконструират структуриран изход (декодер). Комбинирането на двете формира *пясъчен часовник*, чиято тясна среда – *стеснение* – принуждава цялата информация да премине през нискоразмерен код; тъй като мрежата трябва да действа или реконструира въз основа на този код, тя запазва единствено най-същественото, което регулира модела и води до компактно, многократно използвано представяне – зародишът на научена абстракция или на състояние на убеждението. Същото стесняване, вградено в иначе равномерна купчина от слоеве, представлява *яка*. Фигурата по-долу илюстрира тези профили.



Фигура 4: Типични форми на мрежата: равномерна поредица от слоеве, стесняваща се фуния, пясъчен часовник с централно стеснение (в оранжево) и равномерна поредица с една тясна „яка“ (в оранжево). Височината на всеки правоъгълник показва ширината на слоя.

Стеснението ограничава *ширината*, а *остатъчните (residual) връзки* решават проблема с *дълбочината*, като добавят входа на даден слой към неговия изход, така че градиентите да достигат до дълбоките слоеве и много дълбоките мрежи да стават обучаеми – принципът, залегнал в основата на ResNet и на дълбоките мрежи за стойности от глава 6. И двете допълват отпадането, отслабването на теглата и нормализацията, споменати по-рано: всяка от тях определя начина, по който се използва капацитетът, а не само неговия обем.⁶

Остатъчна (residual) връзка: пряк път, който добавя входа на слой към неговия изход, така че слой трябва да научи само корекция; тя поддържа градиентите силни през дълбоки мрежи и е това, което прави много дълбоките модели обучаеми.

5.2.5 Стабилност на обучението и диагностиката

Две стабилизиращи средства от обсъждането на „смъртоносната троица“ – целевата мрежа и буферът за възпроизвеждане – вършат по-голямата част от работата по предотвратяване на разходимостта при обучението, основано на оценка на стойностите. Още две се срещат специално в невронните решавачи на игри по-нататък. *Извадка от резервоар* поддържа фиксиран по размер, равномерно случаен образец от всички данни, видени досега, което е важно, когато дадено количество трябва да бъде осреднено върху цялата история на обучението, а не само върху последните данни; Deep CFR го използва, така че неговата стратегическа мрежа да се обучава върху неизмествена извадка през всички итерации (раздел 5.3).⁷ *Намаляване на дисперсията* – чрез изваждане на базова линия от взетите извадкови стойности или използване на контролни променливи⁸ – намалява шума, който извадката внася в градиента, така че мрежата да се сближава с по-малко извадки. Нормализиране на входа и разумна инициализация на теглата, споменати по-рано, до-

⁶He, K., Zhang, X., Ren, S. & Sun, J. (2016). “Deep Residual Learning for Image Recognition.” *CVPR*, 770–778. DOI 10.1109/CVPR.2016.90.

⁷Brown, N., Lerer, A., Gross, S. & Sandholm, T. (2019). “Deep Counterfactual Regret Minimization.” *ICML*.

⁸Schmid, M., Burch, N., Lanctot, M., Moravčík, M., Kadlec, R. & Bowling, M. (2019). “Variance Reduction in Monte Carlo Counterfactual Regret Minimization (VR-MCCFR) for Extensive Form Games Using Baselines.” *AAAI* 33, 2157–2164.

пълват списъка: и двете поддържат активациите и градиентите в използваем диапазон още от първата стъпка.

Извадка от резервоар: метод, който поддържа извадка с фиксиран размер, при която всеки елемент, видян досега, има еднаква вероятност да бъде запазен, така че текущата средна остава неизместена, без да се съхраняват всички данни.

Дори и с тези мерки на място, невронният решаващ се проваля по-често тихо, отколкото шумно, така че четенето на диагностиката е от значение. Загуба, която се променя, докато оценителният показател – експлоатируемост за решаващ – застива, е признак, че мрежата *се обучава, но още не е достигнала сходимост*: обучението работи, но бюджетът е твърде малък, за да може средната стратегия да се установи, което показаха мрежите от експериментите в тази глава при Ледюк на няколко хиляди обхождания. Обратният и по-опасен случай е, когато обучението изобщо не протича, въпреки че изглежда така: при изследването едносимволна грешка в рутина на OpenSpiel остави мрежите за предимство необучени, но програмата завърши изпълнението си и произведе правдоподобен резултат, като експлоатируемостта просто замръзна около 1,69 независимо от броя итерации (равномерно случайната стратегия има 2,37). Дори след поправката опитът с мрежа 64×64 остава на 1,70 във всяка контролна точка, т.е. на същото ниво, затова тази глава не разчита на неговите числа до повторното му изпълнение. Урокът е да се потвърди, че загубата действително се променя и че оценителният показател реагира, преди да се вярва на който и да е резултат – обучение, което тихо не прави нищо, изглежда точно като труден проблем, освен ако кривите не бъдат проверени.

5.3 Невронни мрежи в CFR

5.3.1 От табличен CFR към апроксимация на функция

Табличният CFR поддържа по две стойности за всяко информационно множество: кумулативно контрафактично съжаление за всяко действие, от което се извежда текущата стратегия чрез напасване на съжалението, както и кумулативна стратегия, чиято текуща средна представлява приближението на равновесието на Наш, което алгоритъмът в крайна сметка връща (Глави 2–4). И двете таблици се разрастват с увеличаването на броя на информационните множества – стената, описана в началото на тази глава.

Замяната им с невронни мрежи е концептуално проста: апроксиматор на функции приема тензор на информационно състояние като вход и предсказва това, което таблицата би съхранявала. Една мрежа може да научи съжаленията – еквивалентно, контрафактичните предимства – които определят текущата стратегия; друга може да научи средната стратегия директно. Тъй като една мрежа обобщава подобни информационни състояния, тя никога не трябва да ги изброява и нейният размер е фиксиран от параметрите ѝ, а не от играта. Методите по-долу се различават основно по това коя от двете таблици апроксимират и как генерират данните за обучението ѝ: Deep CFR научава предимства от извадкови обхождания на дървото, докато NFSP научава средната стратегия от игра срещу себе си без изобщо да обхожда дървото.⁷

5.3.2 Deep CFR и неговите варианти с една мрежа

Deep CFR (Brown и др., 2019) е прекият невронен аналог на MCCFR. При всяка итерация той обхожда дървото на играта чрез външно вземане на проби и във всяко информационно множество, което посещава, изчислява извадковото контрафактивно предимство на всяко действие – колко по-добро е било това действие от текущата стратегия средно. Тези двойки (информационно състояние, предимство) се записват в резервоарен буфер и *мрежа за предимства*, по една за всеки играч, се обучава да ги предсказва; текущата стратегия във всяко информационно множество след това се извежда от предсказаните предимства на мрежата чрез напасване на съжалението, точно както би била използвана таблицата. Отделна *стратегическа мрежа*, обучавана от собствен резервоарен буфер, научава средната стратегия през всички итерации – приближението на равновесието на Наш, което се връща в края.

Единственото уравнение, което отличава Deep CFR от табличния MCCFR, е целевата функция на мрежата за предимства: средноквадратична грешка между нейната прогноза и извадковите предимства,

$$L(\theta) = \frac{1}{|B|} \sum_{(I, \tilde{a}) \in B} \|f_{\theta}(I) - \tilde{a}\|^2,$$

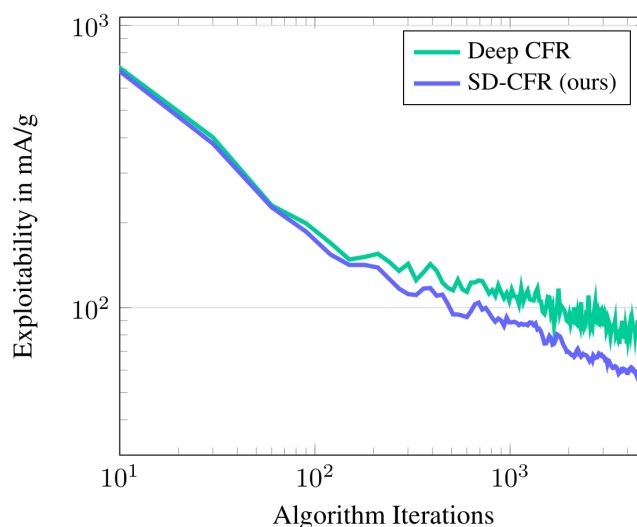
където B е партида, изтеглена от резервоара, I тензор на информационно състояние, $\tilde{a}$ векторът на извадковите предимства за неговите действия и f_{θ} мрежата за предимства (на практика пробите се претеглят според итерацията, отразявайки линейния CFR). Всичко останало – обхождането на дървото, стъпката на напасване на съжалението, осредняването – се запазва непроменено спрямо табличния алгоритъм.

Последваха две усъвършенствания. *Single Deep CFR* премахва отделната стратегическа мрежа: пази мрежата за предимства от всяка итерация и възстановява средната стратегия пряко от тях, с което отпада грешката от приближаването на средната стратегия. В Ледюк то достига същата или малко по-ниска експлоатируемост от Deep CFR, а в пряк двубой в по-голямата игра 5-Flop Hold'em го побеждава.⁹ *DREAM* прави метода безмоделен, т.е. той вече не изисква перфектен симулатор на играта: вместо външно вземане на проби използва извадка по резултати (по една траектория на обхождане) и добавя научена базова линия като контролна променлива, която поема допълнителната дисперсия – същата идея за намаляване на дисперсията от раздел 5.2.¹⁰ *ESCHER* отива още по-далеч: премахва изцяло претеглянето по значимост (importance sampling), като оценява съжалението чрез научена функция на стойността на историята, и в пряк двубой в „тъмен шах“ (dark chess) побеждава DREAM и NFSP в над 90% от игрите.¹¹

⁹Steinberger, E. (2019). “Single Deep Counterfactual Regret Minimization.” *arXiv:1901.07621*.

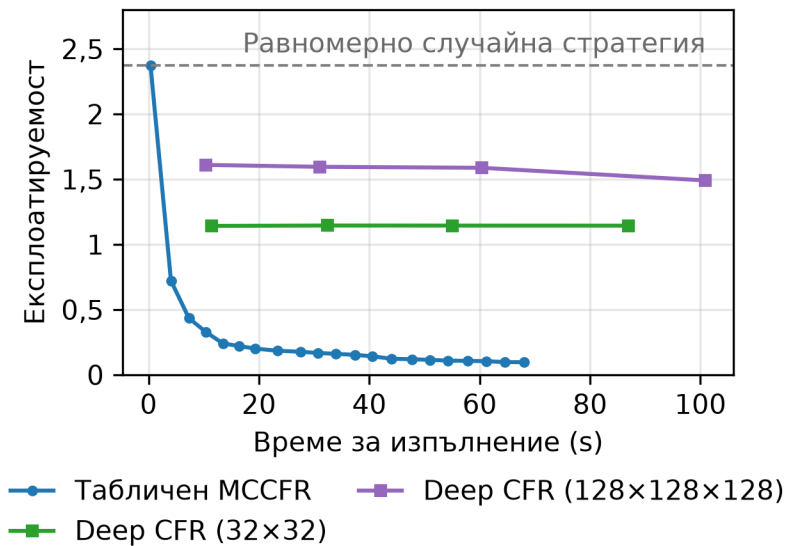
¹⁰Steinberger, E., Lerer, A. & Brown, N. (2020). “DREAM: Deep Regret Minimization with Advantage Baselines and Model-free Learning.” *arXiv:2006.10410*.

¹¹McAleer, S., Farina, G., Lanctot, M. & Sandholm, T. (2023). “ESCHER: Eschewing Importance Sampling in Games by Computing a History Value Function to Estimate Regret.” *ICLR*; *arXiv:2206.04122*.



Фигура 5: Single Deep CFR (SD-CFR) достига същата или малко по-ниска експлоатируемост от Deep CFR в Ледюк (в хилядни от анте на игра, mA/g), без да обучава мрежа за средната стратегия. Фигурата е от статията на Steinberger (2019), фиг. 1a, и не представя експеримент от настоящата работа.

В Ледюк изследването потвърди както обещанието, така и уловката. Deep CFR се обучаваше – загубите на мрежите за предимства се променяха (и растяха, защото целите се претеглят с номера на итерацията) – но на 120 итерации експлоатируемостта му остана над 1,1 при всички размери на мрежата (1,14 в най-добрия случай), срещу 0,097 за табличния MCCFR при сравнимо време за изпълнение. Това само по себе си не е дефект: Ледюк е достатъчно малък, за да може табличният подход да спечели категорично, а на Deep CFR е необходим много по-голям бюджет от извадки, за да се доближи до равновесие на Наш (тук с 40 обхождания на итерация срещу 1500 в настройката на Steinberger за Ледюк). Неговото предимство се проявява едва когато играта е твърде голяма, за да съществува таблица – режимът, който раздел 5.3.4 уточнява.



Фигура 6: Експлоатируемост на Deep CFR (32×32 и 128×128×128) и на табличния MCCFR в Ледюк спрямо времето за изпълнение; пунктирната линия е равномерно случайната стратегия.

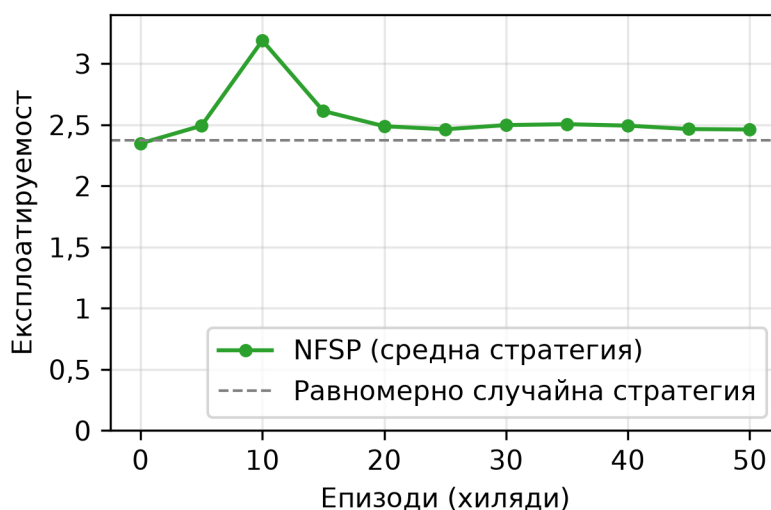
5.3.3 NFSP – Neural Fictitious Self-Play

NFSP – Neural Fictitious Self-Play (Heinrich & Silver, 2016) – достига приблизително равновесие на Наш от страната на обучението с подкрепление, без никога да обхожда дървото на играта. Всеки играч поддържа две мрежи: *мрежа за най-добър отговор* (DQN), която научава алчен най-добър отговор на текущото поведение на опонента, и *мрежа на средната стратегия*, която чрез обучение с учител върху собствените минали действия на играча при игра с най-добрия отговор научава средното по времето на тези най-добри отговори. Средната стратегия е тази, която клони към равновесие на Наш – същото осредняване, което CFR извършва, но постигнато чрез игра, а не чрез съжаление.

Двете мрежи са свързани чрез *антиципиращия параметър* η : в началото на всеки епизод агентът избира с вероятност η да играе с мрежата за най-добър отговор, а в останалите случаи – с мрежата на средната стратегия. Играта с най-добър отговор продължава да генерира свежи данни от разпределението, които средната стратегия усвоява, докато средната стратегия осигурява стабилния опонент, спрямо когото се изчислява най-добрият отговор. Не е необходим резервоар от обхождания на дървото – данните идват единствено от епизоди на игра срещу себе си.

Цената е ниската ефективност по отношение на данните. В играта Ледюк изследването стартира NFSP за десетки хиляди епизоди и наблюдава колебание на експлоатируемостта около 2,5, близо до нивото на равномерно случайната стратегия (2,37), вместо спад, тъй като достигането на използваемо равновесие дори в тази малка игра изисква порядъка на един милион епизоди или повече – собственият пример на OpenSpiel за Ледюк използва още повече епизоди. NFSP се мащабира до големи игри, където безмоделната му простота е предимство, но в учебна игра като

Ледюк той се сближава значително по-бавно от табличния CFR или Deep CFR.¹²



Фигура 7: Експлоатируемост на NFSP в Ледюк в хода на 50 000 епизода обучение: кривата остава около нивото на равномерно случайната стратегия (пунктир, $\approx 2,37$).

5.3.4 Компромиси: кога невронният CFR е от полза

В обобщение: каква е реалната полза от невронните мрежи в CFR и каква е цената им? Ползата е обобщаване и ограничена памет – една мрежа споделя структура между сходни информационни множества и съхранява стратегия във фиксиран брой параметри, така че изчисляването на равновесие вече не изисква изброяване или дори посещение на всяко информационно множество. Цената е висок разход на итерация и по-слаби гаранции: всяка итерация обучава мрежи вместо да увеличава броячи, сходимостта е по-шумна, а чистото монотонно поведение на табличен CFR става приблизително. Резултатите от изследването в Ледюк показват тази страна на компромиса при малки игри: когато таблицата се побира в паметта, тя е по-ефективна.⁷

Компромисът се променя при увеличаване на мащаба. С разрастването на играта табличните таблици в крайна сметка изобщо не се побират в паметта, докато отпечатъкът на мрежата остава непроменен; след този преход невронната методика не е просто конкурентна, а единствената възможна. Затова Deep CFR и неговите разновидности са важни за пълномащабен покер, въпреки че губят в Ледюк.

Едно структурно ограничение определя къде се прилагат невронните методи, базирани на съжаление. CFR изчислява контрафактни стойности чрез разсъждения върху цялото поддърво под едно информационно множество, което е евтино, когато дървото е *широко, но плитко* – покерът има огромен брой **раздавания**, но завършва след няколко рунда на залагане. В игри, които са *дълбоки* – със стотици последователни хода, както в една решетъчна среда – това разсъждение

¹²Heinrich, J. & Silver, D. (2016). "Deep Reinforcement Learning from Self-Play in Imperfect-Information Games." *arXiv:1603.01121*.

става непосилно скъпо и безмоделните методи за игра срещу себе си от раздел 5.4 се превръщат в по-естественото решение. Семейството, базирано на съжаление, продължава да расте в своята ниша – NeuRD, например, променя с един ред обновяването по градиента на стратегията така, че то да следва репликаторната динамика; в табличния случай методът е еквивалентен на softmax CFR¹³ – но неговата най-добра област на приложение остават широките, плитките игри с непълна информация.

5.4 Други невронни приложения

5.4.1 Четирите водещи фамилии алгоритми

Раздел 5.3 представи един от начините за прилагане на невронни мрежи в игри с непълна информация – чрез приближаване на CFR. Това е едно от четирите големи семейства, по които в тази глава се подреждат съвременните методи, като изборът между тях до голяма степен зависи от съответствието им с мащаба на играта, структурата на дървото и броя на участниците.¹⁴ Четирите са методи, *базирани на съжаление* (Deer CFR, DREAM, NeuRD), от раздел 5.3; *популационни* методи (PSRO и неговите разновидности, включително NFSP); методи, *съчетаващи търсене и обучение с подкрепление* (ReBeL, Student of Games); и *игра срещу себе си без използване на модел* (PPO, DQN, MAPPO, QMIX). Таблицата ги представя; следващите раздели разглеждат трите, които все още не са обхванати.

Таблица 1: Семейства невронни методи за игри с непълна информация: основна идея, приложимост и цена.

Семейство	Основна идея	Най-подходящо за	Цена
Базиран на съжаление (Deer CFR, DREAM, NeuRD)	Приближаване на CFR с невронни мрежи	Широки, плитките игри; неексплоатируемост	Висока
Популационен (PSRO, NFSP, XFP)	Най-добър отговор на нарастваща популация от противници	С n играчи; моделиране на противника	Средна
Търсене + RL (ReBeL, Student of Games)	Научена функция на стойността плюс търсене по време на игра	Игри с високи залози; изчисления по време на игра	Висока
Игра срещу себе си без използване на модел (PPO, DQN, MAPPO, QMIX)	Оптимизиране на награда чрез игра срещу себе си	Пространствени, с дълъг хоризонт (решетки)	Ниска–средна

¹³Hennes, D. et al. (2020). “Neural Replicator Dynamics: Multiagent Learning via Hedging Policy Gradients.” AAMAS, 492–501; arXiv:1906.00190.

¹⁴Rudolph, M. et al. (2026). “Reevaluating Policy Gradient Methods for Imperfect-Information Games.” ICLR; arXiv:2502.08938.

5.4.2 Популационни методи: PSRO, NFSP, XFP

Популационните методи преформулират **намирането на равновесие** като итеративен турнир. Започвайки от малък набор от **стратегии**, се формира *мета-игра* чрез изиграване на членовете на набора един срещу друг; след това се обучава нова **стратегия** като приблизителен **най-добър отговор** на текущата **популация**, добавя се обратно към набора и процесът се повтаря. Това е идеята зад PSRO (Policy-Space Response Oracles) и неговите мащабируеми варианти като Pipeline PSRO; фиктивната **игра срещу себе си**, включително NFSP от раздел 5.3, е специалният случай, при който **най-добрият отговор** е насочен към средното по времето на набора. Тъй като **популацията** представлява изричен набор от противници, а не една **подвижна цел**, тези методи естествено излизат извън рамките на игрите с нулева сума за двама играчи и обхващат сценариите с n играчи и моделиране на противника, които са предмет на тази дисертация; популационно обучение от този вид е в основата и на „лигата“ от агенти, с която AlphaStar, първоначално обучен чрез подражание на човешки игри, достига нивото на гросмайстор в StarCraft II.⁴ Цената е поддържането и съхраняването на множество агенти, а гаранциите за **равновесие** са по-слаби от тези на CFR.¹⁵

5.4.3 Търсене + RL: ReBeL и Student of Games

Методите търсене плюс обучение с подкрепление представляват идеята на AlphaZero, адаптирана към скрита информация. В игра с пълна информация може да се извършва търсене напред от текущото състояние, тъй като то е известно; при непълна информация търсещият играч не знае действителното състояние, затова търсенето трябва да обхваща разпределение на вероятностите върху възможни състояния. ReBeL (Brown и др., 2020)¹⁶ прави това прецизно чрез *публичното състояние на убежденията* (PBS) – разпределение на вероятностите върху частната информация на играчите, предвид всичко публично – и изпълнява CFR върху малка, ограничена по дълбочина под-игра по време на игра, като научена мрежа за стойности предоставя стойностите на листата. Student of Games (Schmid и др., 2023; в предварителната версия – Player of Games)¹⁷ обобщава този подход в един алгоритъм, който обработва както пълна, така и непълна информация. Тези системи са в основата на най-силните резултати в покера (ReBeL достига свръхчовешко ниво в безлимитния тексаски Холдем за двама играчи), но са най-сложни за изграждане и зависят от изчисления, извършвани *по време* на игра, а не само по време на обучение; те са предмет на Глава 6, така че тази секция само ги позиционира в общата картина.

5.4.4 Игра срещу себе си без използване на модел: PPO, DQN, MAPPO, QMIX

Методите за игра срещу себе си без използване на модел изцяло игнорират дървото на играта и оптимизират директно сигнала за награда, усъвършенствайки стратегия от епизоди на игра срещу

¹⁵Lanctot, M. et al. (2017). “A Unified Game-Theoretic Approach to Multiagent Reinforcement Learning.” *NeurIPS* (PSRO).

¹⁶Brown, N., Bakhtin, A., Lerer, A. & Gong, Q. (2020). “Combining Deep Reinforcement Learning and Search for Imperfect-Information Games.” *NeurIPS* 33; arXiv:2007.13544.

¹⁷Schmid, M. et al. (2023). “Student of Games: A unified learning algorithm for both perfect and imperfect information games.” *Science Advances* 9(46), eadg3256. DOI 10.1126/sciadv.adg3256.

собствени копия. Те са сред най-използваните методи в многоагентното обучение с подкрепление: PPO¹⁸ и DQN¹⁹ за едноагентното ядро и многоагентните разширения MAPPO²⁰ и QMIX²¹ за кооперативни сценарии. Тъй като никога не обмислят поддържането под дадено състояние, те не се затрудняват от дълбочината, която пречи на методите, основани на съжалението, което ги прави естествения избор за пространствени игри с дълъг хоризонт – решетъчни среди, Pommerman и подобни. За да се справят с частичната наблюдаемост, те използват рекурентните архитектури и архитектурите с внимание от раздел 5.2, предоставяйки на агента памет за това, което мъглата е скрила. Наивната игра срещу себе си с алгоритъм за един агент може да се върти в цикъл и да остане силно експлоатируемо. Затова по-новите безмоделни методи променят динамиката на обучението (NeuRD,¹³ R-NaD в DeepNash,²² магнитно огледално спускане, MMD²³), така че играта да се доближава до равновесие. Масово сравнение в пет игри с нулева сума за двама играчи показва, че специализираните методи NFSP, PSRO, ESCHER и R-NaD не превъзхождат по експлоатируемост общите методи с градиент на стратегията като PPO и MMD.¹⁴ Затова експлоатируемостта на всеки безмоделен агент трябва да се измерва, а не да се предполага.

¹⁸Schulman, J., Wolski, F., Dhariwal, P., Radford, A. & Klimov, O. (2017). “Proximal Policy Optimization Algorithms.” arXiv:1707.06347.

¹⁹Mnih, V. et al. (2013). “Playing Atari with Deep Reinforcement Learning.” arXiv:1312.5602; Mnih, V. et al. (2015). “Human-level control through deep reinforcement learning.” *Nature*, 518(7540), 529–533.

²⁰Yu, C. et al. (2022). “The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games.” *NeurIPS Datasets and Benchmarks*; arXiv:2103.01955.

²¹Rashid, T. et al. (2018). “QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning.” *ICML*; arXiv:1803.11485.

²²Perolat, J. et al. (2022). “Mastering the game of Stratego with model-free multiagent reinforcement learning.” *Science* 378, 990–996. DOI 10.1126/science.add4679.

²³Sokota, S. et al. (2023). “A Unified Approach to Reinforcement Learning, Quantal Response Equilibria, and Two-Player Zero-Sum Games.” *ICLR*; arXiv:2206.05825.